

PotionCheck

Project Flow Document

AI Ingredient Intelligence Scanner & Personalized Food Analysis Platform

Project Name	PotionCheck
Version	1.0
Document Type	Project Flow Document
Prepared By	Nishant (2310993892) Mohit (2310993967)
Institution	Chitkara University Institute of Engineering & Technology, Rajpura
Program	BE-CSE (Artificial Intelligence) - Semester 6
Date	May 2026
SRS Reference	PotionCheck SRS v1.0 (May 2026)
Status	Final Submission

1. Introduction

PotionCheck is a web-based, AI-powered ingredient intelligence scanner developed as a B.Tech Semester 6 project at Chitkara University, Rajpura, Punjab. This Project Flow Document describes the end-to-end lifecycle of the project - from planning and requirements through development, testing, and delivery. The document serves as the master reference for how the system is built, how food data flows through the platform, and how each development phase connects to the functional requirements defined in SRS v1.0.

1.1 Purpose

This document provides:

- A clear overview of all project phases with timelines and milestones
- The complete data and process flow across all system layers
- Traceability from functional requirements to implementation phases
- AI analysis workflow and system architecture diagrams
- Roles and responsibilities of all team members

1.2 System Overview

PotionCheck solves the problem of confusing packaged food labels. The platform allows users to scan a barcode, upload a food label image, paste ingredient text, or select a produce-style flow and then receive a simple health report. The report includes safety score, health score, verdict, flagged ingredients, nutrition observations, personalized warnings, and recommendation. The system is powered by a React/Vite frontend, FastAPI backend, SQLite, Redis, Open Food Facts, OCR, Groq AI, Ollama fallback, and JWT authentication.

2. Project Phases & Timeline

The project is structured into seven sequential phases spanning January 2026 to May 2026. Each phase has defined entry criteria, deliverables, and exit criteria.

#	Phase	Period	Owner	Key Deliverable
1	Planning & Requirements	Jan 2026	All Members	SRS v1.0, Project Charter
2	System Design & Architecture	Feb 2026	All Members	Architecture Diagram, DB Schema, API Design
3	Backend Development	Feb - Mar 2026	Nishant	FastAPI API, Auth, DB, AI/OCR Integration
4	Frontend Development	Mar - Apr 2026	Mohit	React UI, Scanner, Analysis, Profile, History
5	AI, OCR & Product Lookup Integration	Apr 2026	Nishant + Mohit	Barcode Lookup, OCR, Groq/Ollama Analysis
6	Testing & QA	Apr - May 2026	All Members	Test Cases, Bug Fixes, Security Checks
7	Documentation & Delivery	May 2026	All Members	Project Report, Flow Doc, Presentation, Submission

3. Detailed Phase Flow

Phase	Planning & Requirements	January 2026	Owner: All Members
1			Key Deliverable: SRS v1.0, Project Charter

- 1
- Conduct kickoff meetings and assign roles: Nishant for backend, database, AI integration, OCR, and deployment; Mohit for frontend, UI flow, testing support, and documentation.
- 2
- Identify stakeholders, target users, and core problems such as confusing food labels, allergies, health conditions, and difficult nutrition terms.
- 3
- Define main analysis modes: barcode scanning, label upload OCR, pasted ingredient text, AI report, profile-based warnings, and scan history.
- 4
- Write and review SRS v1.0 with functional requirements, non-functional requirements, assumptions, constraints, and known limitations.
- 5
- Set up GitHub repository, Python backend, React frontend, Docker configuration, and environment variable structure.
- 6
- Finalize Project Charter - scope, milestones, team responsibilities, and risk register.

Phase	System Design & Architecture	February 2026	Owner: All Members
2			Key Deliverable: Architecture Diagram, DB Schema, API Design

- 1
- Design three-layer architecture: React/Vite SPA (Frontend) -> FastAPI (Backend) -> SQLite, Redis, Open Food Facts, OCR, and AI services.
- 2
- Design product analysis pipeline: User Input -> Scanner/OCR/Text Parser -> Product Data -> User Profile -> AI Analysis -> Report UI -> History.
- 3
- Design REST API contract - endpoints, request/response schemas, JWT auth flow, and error codes.
- 4
- Design database schema: Users, UserProfile, ScanHistory, and Analysis tables.
- 5
- Design fallback architecture - Groq primary AI -> Ollama fallback -> local rule-based analysis.
- 6
- Document technology decisions: React, Tailwind CSS, FastAPI, SQLite, Redis, Open Food Facts, Tesseract OCR, Groq, Ollama, Docker, Render.

Phase	Backend Development	February - March 2026	Owner: Nishant
3			Key Deliverable: FastAPI API, JWT Auth, Database, AI and OCR Integration

- 1
- Initialize FastAPI backend project - configure CORS, environment variable loading, database initialization, and router structure.
- 2
- Implement JWT-based authentication - user registration, login, refresh token, logout, and current user endpoint.
- 3
- Hash all user passwords with bcrypt before storage; validate email format and password length on registration.
- 4
- Implement SQLite connection and SQLAlchemy async data models: User, UserProfile, ScanHistory, and Analysis.
- 5
- Build scanner endpoints for barcode lookup, barcode analysis, image upload, OCR extraction, and pasted text parsing.
- 6
- Integrate Open Food Facts API and Redis caching for repeated product lookups.
- 7
- Implement AI analysis service using Groq as primary provider, Ollama fallback, and local rule-based fallback.
- 8
- Implement scan history APIs with pagination, verdict filtering, search, single deletion, and all-scan deletion with email confirmation.
- 9
- Implement health check endpoint for backend, database, Redis, and Open Food Facts status.

Phase	Frontend Development	March - April 2026	Owner: Mohit
4			Key Deliverable: React UI, Scanner, Analysis Report, Profile, History

- 1 Initialize React + Vite + Tailwind CSS project - configure build pipeline and environment variables.
- 2 Build Landing Page - application overview, visual background, navigation, and call-to-action buttons.
- 3 Build Profile page - registration, login, health profile, allergies, health conditions, and diet type.
- 4 Build Scanner page - live barcode scanner, manual barcode entry, label image upload, pasted text workflow, and status feedback.
- 5 Build Analysis page - product details, score meter, verdict, nutrition breakdown, flagged ingredients, all ingredient explanations, AI summary, and recommendation.
- 6 Build History page - saved scan records, local recent history, filters, searching, and deletion actions.
- 7 Build Chatbot page for ingredient and food-related questions.
- 8 Implement responsive layout, animated background, navbar, toast notifications, transitions, and mobile-friendly interactions.
- 9 Centralize API communication in service modules for scanner, profile, auth, analysis, history, and chatbot.

Phase	AI, OCR & Product Lookup Integration	April 2026	Owner: Nishant + Mohit
5			Key Deliverable: Barcode Lookup, OCR, Groq/Ollama Analysis

- 1 Integrate barcode flow end-to-end: frontend scanner -> backend barcode endpoint -> Open Food Facts -> cache -> analysis report.
- 2 Validate product data handling when ingredients, nutriments, brand, category, or image fields are missing.
- 3 Integrate label upload flow end-to-end: image upload -> backend validation -> OCR extraction -> editable text -> AI analysis.
- 4 Integrate pasted text flow end-to-end: user text -> parser -> ingredient list preview -> custom analysis.
- 5 Validate AI fallback flow: Groq response, Groq failure, Ollama fallback, and local rule-based analysis.
- 6 Normalize AI JSON output so frontend receives stable fields for scores, verdicts, warnings, ingredients, summary, and recommendation.
- 7 Perform cross-browser testing - Chrome, Firefox, Edge, Safari on desktop and mobile viewports.

Phase	Testing & QA	April - May 2026	Owner: All Members
6			Key Deliverable: Test Cases, Bug Fixes, Security Checks

- 1 Execute test cases across Authentication, Profile, Scanner, OCR Upload, Text Analysis, AI Analysis, Report UI, History, Chatbot, Health Check, and Deployment modules.
- 2 Test invalid inputs including duplicate email, wrong password, invalid barcode, empty ingredient text, non-image upload, and oversized image upload.
- 3 Validate JWT token issuance, refresh handling, logout, protected endpoints, and rejection of unauthorized access.
- 4 Conduct CORS, file-upload, environment variable, and error-handling checks.
- 5 Log defects; triage by severity; fix all critical and high-severity bugs.
- 6 Review UI on desktop and mobile screens to ensure content is clear, readable, and not overlapping.

Phase	Documentation & Delivery	May 2026	Owner: All Members
7			Key Deliverable: Project Report, SRS, Flow Document, Test Cases, Presentation, Final Submission

- 1 Finalize SRS document with table of contents, functional requirements, non-functional requirements, data requirements, and appendices.
- 2 Complete Project Charter - scope, milestones, resource table, risk register, and sign-off section.
- 3 Write Project Flow Document - this document; covers all phases, data flows, AI/OCR workflows, architecture, traceability, and roadmap.
- 4 Compile full test case document with module-wise test cases and pass/fail/pending status.
- 5 Prepare final project presentation and live demo for academic evaluation committee.
- 6 Submit all project artifacts: SRS, Charter, Flow Document, Test Cases, Project Report, Source Code, and Presentation.

4. System Architecture & Data Flow

4.1 Three-Layer Architecture

Layer	Technology	Description
Layer 1 - Frontend (SPA)	React + Vite + Tailwind CSS	Responsive single-page application. Communicates with backend through API calls only.
Layer 2 - Backend (API)	FastAPI + Uvicorn	Handles authentication, user profile, scanner, OCR, AI analysis, history, chatbot, and health checks.
Layer 3 - Data & External	SQLite + Redis + Open Food Facts + Groq + Ollama	Stores users and reports, caches product data, fetches barcode products, and generates analysis.

4.2 Request Flow - Barcode Product Analysis

- 1 User scans a barcode using camera, uploads barcode image, or enters barcode manually on the Scanner page.
- 2 Frontend validates barcode length and format before sending the request.
- 3 Frontend sends POST /api/scanner/barcode/analyze with barcode and optional profile data.
- 4 Backend checks Redis cache; if cache is empty, it fetches product data from Open Food Facts.
- 5 Backend returns 404 if product is not found and 422 if ingredient data is missing.
- 6 Backend combines product ingredients, nutriments, product context, and user profile.
- 7 AI service analyzes using Groq; if unavailable, it uses Ollama; if unavailable, it uses local rules.
- 8 Backend saves scan history and analysis result in SQLite when applicable.
- 9 Backend returns JSON response with product details, scores, verdict, warnings, ingredients, and recommendation.
- 10 Frontend renders the analysis report with score meter, cards, warnings, and easy explanations.

4.3 Authentication Flow

- 1 User fills Sign Up form with full name, email, and password; frontend validates basic form input.
- 2 Frontend sends POST /api/auth/register to the backend.
- 3 Backend validates email uniqueness and password length.
- 4 Backend hashes password using bcrypt and stores the new user in SQLite.
- 5 On login, frontend sends POST /api/auth/login with credentials.
- 6 Backend verifies credentials against hashed password.
- 7 Backend issues signed JWT access token and HTTP-only refresh cookie.
- 8 Frontend sends the access token for protected API requests.
- 9 Backend validates JWT on protected endpoints and rejects invalid or expired tokens with HTTP 401.
- 10 User can refresh token with POST /api/auth/refresh and log out with POST /api/auth/logout.

4.4 OCR and Custom Text Analysis Flow

- 1 User uploads a food label image or pastes ingredient text on the Scanner page.
- 2 For image upload, backend validates image type and maximum 10 MB size.
- 3 Backend stores the image temporarily and runs OCR to extract ingredient text.
- 4 Frontend displays extracted text so user can review and correct it.
- 5 For pasted text, backend cleans and parses the ingredient list.
- 6 User sends final ingredient text for AI analysis.
- 7 Backend combines text with profile and available nutrition data.
- 8 AI service returns score, verdict, ingredient explanations, warnings, summary, and recommendation.

5. AI Analysis Workflow

PotionCheck operates through a structured ingredient intelligence workflow. The workflow combines user input, product data, profile personalization, OCR, AI reasoning, fallback safety, storage, and report rendering.

#	Process	Description
1	User Input	Barcode, image label, pasted text, or saved analysis is selected by the user.
down		
2	Input Processor	Frontend and backend validate barcode, image, or text input.
down		
3	Product Data Layer	Backend fetches product data from Open Food Facts or extracts text using OCR.
down		
4	Profile Personalizer	Backend applies allergies, health conditions, and diet type.
down		
5	AI Provider Chain	Groq primary analysis, Ollama fallback, local rule fallback.
down		
6	Result Normalizer	Backend converts AI output into stable JSON fields.
down		
7	Storage Layer	ScanHistory and Analysis are stored in SQLite when required.
down		
8	Response to UI	Frontend renders score, verdict, flagged ingredients, nutriments, and recommendation.

Left flow: User Input -> Scanner/OCR/Text Processing -> Product Data -> AI Analysis

Right flow: Profile Personalization -> Result Normalization -> Storage -> Report UI

6. Functional Requirements Traceability

All functional requirements from PotionCheck SRS v1.0 are mapped to their implementation phases below.

Req ID	Module	Description	Phase	Priority
REQ-01	Authentication	User registration with unique email, name, and password	Phase 3	Must
REQ-02	Authentication	Password length and email format validation	Phase 3	Must
REQ-03	Authentication	Password hashing with bcrypt before DB storage	Phase 3	Must
REQ-04	Authentication	JWT access token and refresh cookie issuance	Phase 3	Must
REQ-05	Authentication	Protected current-user and user-specific endpoints	Phase 3	Must
REQ-06	Profile	Save allergies, health conditions, and diet type	Phase 3 & 4	Must
REQ-07	Scanner	Manual barcode entry and live camera scanning	Phase 4 & 5	Must
REQ-08	Scanner	Uploaded barcode image support	Phase 4 & 5	Should
REQ-09	Product Lookup	Fetch product data from Open Food Facts	Phase 3 & 5	Must
REQ-10	Product Lookup	Cache product lookup result using Redis	Phase 3 & 5	Should
REQ-11	OCR Upload	Accept image upload and reject invalid or oversized files	Phase 3 & 5	Must
REQ-12	OCR Upload	Extract ingredient text from uploaded label image	Phase 3 & 5	Must
REQ-13	Text Analysis	Clean and parse pasted ingredient text	Phase 3 & 4	Must

Req ID	Module	Description	Phase	Priority
REQ-14	AI Analysis	Analyze ingredients using Groq primary provider	Phase 3 & 5	Must
REQ-15	AI Analysis	Fallback to Ollama and local rule-based analysis	Phase 3 & 5	Must
REQ-16	AI Analysis	Return safety score, health score, verdict, flagged ingredients, and recommendation	Phase 3 & 5	Must
REQ-17	Report UI	Display product details, nutrition, warnings, explanations, and score meter	Phase 4 & 5	Must
REQ-18	Report UI	Show easy-language explanation for why each concern matters	Phase 4 & 5	Must
REQ-19	History	Store and retrieve authenticated user's scan history	Phase 3 & 4	Must
REQ-20	History	Search, filter, delete one scan, and delete all scans with confirmation	Phase 3 & 4	Should
REQ-21	Chatbot	Provide ingredient and food question support through chatbot service	Phase 3 & 4	Should
REQ-22	Health Check	Expose backend health endpoint with dependency status	Phase 3	Must
REQ-23	Deployment	Support Docker Compose and Render deployment	Phase 7	Must
REQ-24	Documentation	Provide SRS, Flow Document, project report, presentation, and source code	Phase 7	Must

7. Roles & Responsibilities

Team Member	Roll No.	Role	Responsibilities
Nishant	2310993892	Backend / AI Integration Lead	FastAPI backend, authentication, database models, scanner APIs, Open Food Facts integration, Groq/Ollama AI analysis, OCR, health checks, deployment configuration.
Mohit	2310993967	Frontend Developer / UI-UX / QA Support	React/Vite frontend, scanner page, analysis report UI, profile and history views, chatbot page, responsive design, API service modules, UI testing, documentation support.

7.1 Communication Protocol

Channel	Purpose	Frequency
GitHub	Source code version control, commits, pull requests, and issue tracking	Daily
WhatsApp / Discord	Team coordination, quick updates, and blocker resolution	Daily
Weekly Review	Progress check, milestone validation, and task re-assignment	Weekly
Academic Supervisor	Faculty sign-off on deliverables and evaluation checkpoints	As required

8. Risk Register & Mitigation

ID	Risk	Impact	Likelihood	Mitigation Strategy
R-01	Groq API downtime or quota limit disrupts AI analysis	High	Medium	Use Ollama fallback and local rule-based analysis for graceful degradation.
R-02	Open Food Facts data is missing or inaccurate	Medium	High	Show clear error and allow pasted text or label upload as alternate input.
R-03	OCR accuracy is poor due to blurry label image	Medium	Medium	Allow user to review and edit extracted text before final analysis.
R-04	Camera permission blocks live barcode scan	Medium	Medium	Provide manual barcode entry and image upload as alternate workflows.
R-05	Users treat the system as medical advice	High	Low	Display educational-use disclaimer and advise professional consultation for serious concerns.
R-06	JWT token or account security issue	High	Low	Use hashed passwords, short-lived tokens, HTTP-only refresh cookies, and protected endpoints.
R-07	SQLite becomes insufficient for heavy production traffic	Medium	Low	Plan PostgreSQL migration for future production scaling.
R-08	Team member unavailability due to academic workload	Medium	Medium	Share documentation, organize code, and review progress weekly.

9. Technology Stack Summary

Layer / Component	Technology	Purpose
Frontend Framework	React 18 + Vite	Single-page application and component architecture
Frontend Styling	Tailwind CSS	Responsive styling and modern UI layout
Frontend State	Zustand	Profile, scanner, history, and app state management
HTTP Client	Axios	Frontend API request handling
Barcode	html5-qrcode	Live camera and image barcode scanning
OCR	Tesseract.js / pytesseract	Ingredient text extraction from label images
Backend Runtime	Python 3 + Uvicorn	Backend application runtime server
Backend Framework	FastAPI	REST API routing, validation, and middleware
Database	SQLite + SQLAlchemy async	Users, profiles, scan history, and analyses
Cache	Redis	Caches product lookup data
AI API	Groq	Primary LLM ingredient analysis provider
AI Fallback	Ollama + local rules	Fallback analysis when Groq is unavailable
External Product API	Open Food Facts	Barcode product lookup and nutrition data
Deployment Target	Docker Compose + Render	Local and cloud deployment support

9.1 System Requirements Summary

Requirement Type	Minimum Specification	Recommended
Development CPU	Intel Core i3 / AMD equivalent, 2.0 GHz	Intel Core i5+ / 2.5 GHz+
Development RAM	4 GB	8 GB
Storage (Dev)	10 GB free, SSD preferred	20 GB SSD

Requirement Type	Minimum Specification	Recommended
Production CPU	2 vCPU cores	4 vCPU cores
Production RAM	4 GB	8 GB
Production Storage	20 GB SSD	40 GB SSD
Network (Client)	5 Mbps	10 Mbps+
Browser (Client)	Chrome, Firefox, Edge, Safari latest	Latest versions
Mobile Client	Android 8+ / iOS 14+	Android 12+ / iOS 16+
Backend Software	Python 3, FastAPI, Uvicorn	Latest compatible versions
Frontend Software	Node.js, npm, Vite	Node.js LTS
Cache	Redis server	Managed Redis for production

10. Future Scope & Roadmap

PotionCheck has significant potential for growth beyond the current academic scope. The following enhancements are planned for future iterations:

ID	Feature	Priority	Description
FR-01	PDF Export	High	Allow users to download complete analysis reports as PDF files.
FR-02	Mobile Application	High	Native iOS and Android apps with camera-first barcode scanning and saved reports.
FR-03	PostgreSQL Migration	Medium	Move from SQLite to PostgreSQL for larger production deployments.
FR-04	Serving Size Analysis	High	Add serving-size based scoring in addition to per-100g nutrition scoring.
FR-05	Multilingual OCR	Medium	Support Hindi and other regional languages for ingredient labels.
FR-06	Nutrition Citations	Medium	Add source references and citations for health and nutrition explanations.
FR-07	Admin Dashboard	Medium	Monitor system health, API errors, upload cleanup, and usage statistics.
FR-08	Allergen Alert Mode	High	Provide stronger alerts for serious allergy matches.
FR-09	Product Comparison	Medium	Compare two products and recommend the healthier choice for the user's profile.